

5.(라즈베리파이)

GPIO 및 센서

IoT 사물인터넷

학습목표

- 파이썬 프로그래밍으로 라즈베리파이의 GPIO를 제어할 수 있다.
- 각종 센서를 라즈베리파이에 연결하여 동작시킨다.

1. GPIO 제어 및 테스트

1. GPIO (General Purpose Input Output) 번호와 기능

Functions	GPIO	Pin	Pin	GPIO	Functions
	3.3V	1	2	5V	
SDA1 (I ² C)	GPIO2	3	4	5V	
SCL1 (I ² C)	GPIO3	5	6	GND	
	GPIO4	7	8	GPIO14	TXD0 (UART)
	GND	9	10	GPIO15	RXD0 (UART)
	GPIO17	11	12	GPIO18	PWM0, CLK (PCM)
	GPIO27	13	14	GND	
	GPIO22	15	16	GPIO23	
	3.3V	17	18	GPIO24	
MOSI (SPI)	GPIO10	19	20	GND	
MISO (SPI)	GPIO9	21	22	GPIO25	
SCLK (SPI)	GPIO11	23	24	GPIO8	CE0 (SPI)
	GND	25	26	GPIO7	CE1 (SPI)
	ID_SD	27	28	ID_SC	
	GPIO5	29	30	GND	
	GPIO6	31	32	GPIO12	PWM0
PWM1	GPIO13	33	34	GND	
FS (PCM), PWM1	GPIO19	35	36	GPIO16	
	GPIO26	37	38	GPIO20	DIN (PCM)
	GND	39	40	GPIO21	DOUT (PCM)

핀번호 읽는 법

1. Board(물리적인 핀번호)
: 왼쪽 위부터 1로 시작
: 오른쪽 아래 40번 핀
2. BCM(프로그램 가능한 핀 번호)
: GPIO + 숫자
3. 특수기능(Functions)
: GPIO 기능 외 특수기능 가능
Ex) OLED SDA,SCL

출력(Output)

: Low -> 0 V / High -> 3.3 V

입력(Input)

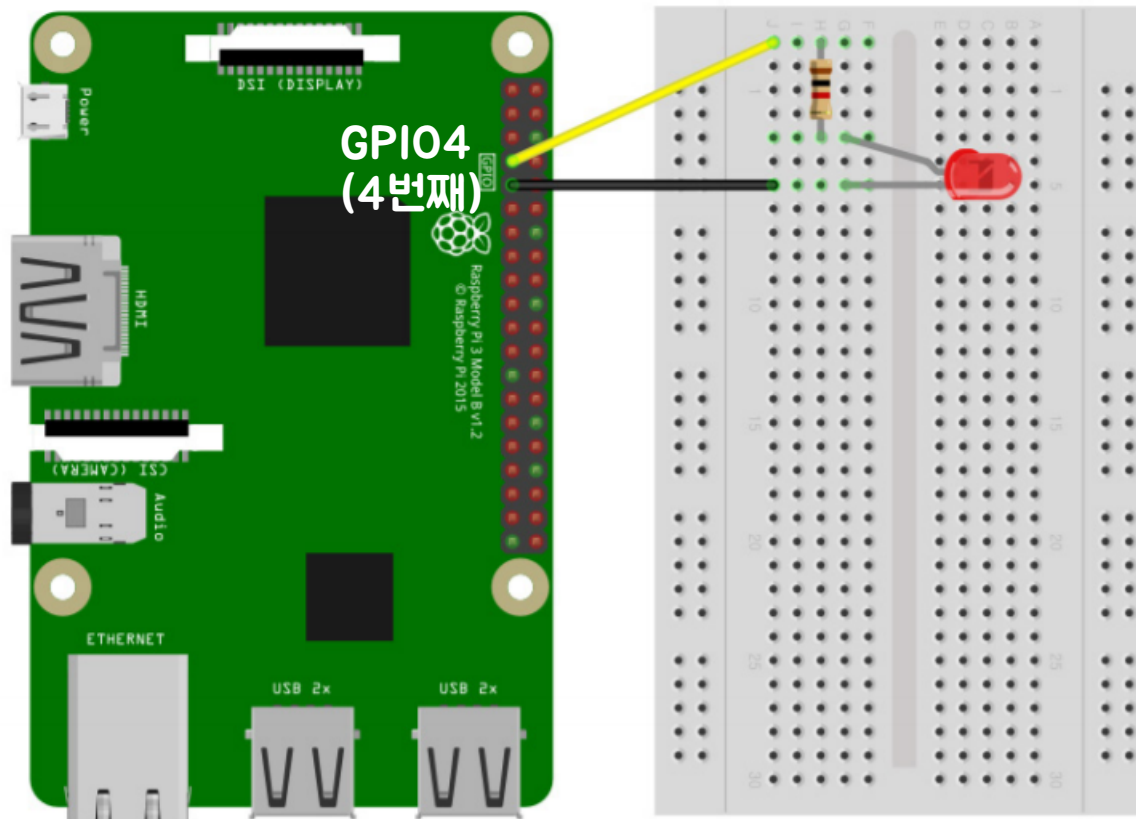
: 3.3 V -> High / 0 V -> Low

마이크로프로세서가 주변장치와 통신하기 위해 범용으로 사용되는 입출력(I/O)포트

1. GPIO 제어 및 테스트

2. LED 실습 : GPIO 출력을 제어하여 LED를 On, Off

1) 브레드보드 연결하기 (저항 220Ω, LED)



RP 4	LED
GPIO 4	+ , 저항
GND	-

1. GPIO 제어 및 테스트

2. LED 실습 : GPIO 출력을 제어하여 LED를 On, Off

2) 파이썬 코드 작성하기

```
1  #-*- coding: utf-8 -*-
2
3  # 필요한 모듈을 불러옵니다.
4  import RPi.GPIO as GPIO
5  import time
6
7  #GPIO핀의 번호 모드 설정
8  #BOARD 2열 헤더 핀의 번호를 순서대로 나열
9  #BCM (Broadcom MPU의 pin 기능번호)
10 GPIO.setmode(GPIO.BCM)
11 #불필요한 warning제거
12 GPIO.setwarnings(False)
13
14 #사용할 GPIO핀의 번호를 선정합니다.(BCM모드)
15 led_pin = 4 #GPIO 4
16 #LED핀의 IN/OUT설정
17 GPIO.setup(led_pin, GPIO.OUT)
```

1. GPIO 제어 및 테스트

2. LED 실습 : GPIO 출력을 제어하여 LED를 On, Off

2) 파이썬 코드 작성하기

```
18
19  #10번 반복
20  for i in range(10):
21      GPIO.output(led_pin, 1) #led On
22      time.sleep(1)           #1초 동안 대기상태
23      GPIO.output(led_pin, 0) #led Off
24      time.sleep(1)
25
26  #GPIO 설정 초기화(모듈 점유 해제)
27  GPIO.cleanup()
```

3) 실행 (Execute)

: 단축키 F5



1. GPIO 제어 및 테스트

2. LED 실습 : GPIO 출력을 제어하여 LED를 On, Off

4) 파이썬 프로그래밍 준비하기 (제작 -> 빌드명령 설정)

: 지니(Geany) 환경설정 - Compile과 Execute에서 python3 확인

#	라벨	명령	작업 폴더	초기화
Python 명령				
1.	Compile	python3 -m py_comp		
2.				
3.	Lint	pep8 --max-line-length		
정규 표현식의 오류:		(.+:)([0-9]+):([0-9]+)		
Independent commands				
1.	make(M)	make		
2.	Make Custom Target...	make		
3.	Make 오브젝트(O)	make %e.o		
4.				
정규 표현식의 오류:				
<i>Note: Item 2 opens a dialog and appends the response to the command.</i>				
명령 실행				
1.	Execute	python3 \"%f\"		
2.				
<i>%d, %e, %f, %p, %l are substituted in command and directory fields, see manual for details.</i>				
				취소(C) 확인(O)

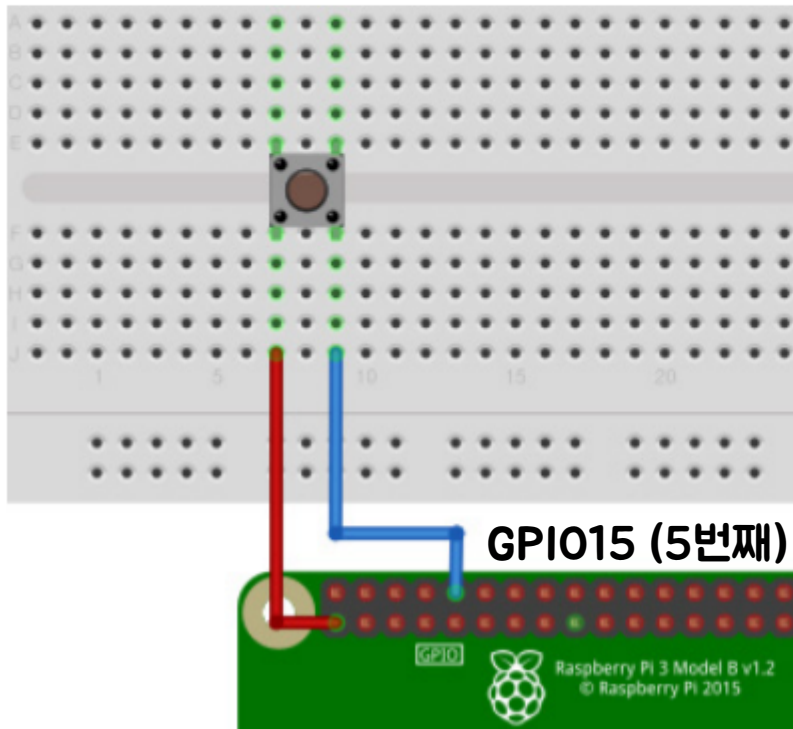
python3 -m py_compile \"%f\"

python3 \"%f\"

1. GPIO 제어 및 테스트

3. 푸시 버튼 스위치 실습 (Polling 방식)

1) 브레드보드 연결하기 (Tack 스위치)



RP 4	스위치
GPIO 15	핀 연결
VCC(3.3V)	핀 연결

1. GPIO 제어 및 테스트

3. 푸시 버튼 스위치 실습 (Polling 방식)

2) 파이썬 코드 작성하기

```
1  #필요한 모듈(라이브러리)을 불러옵니다.
2  import RPi.GPIO as GPIO
3  import time
4
5  #GPIO 모드, 번호 세팅, 불필요한 warning제거
6  GPIO.setmode(GPIO.BCM)
7  GPIO.setwarnings(False)
8  button_pin = 15
9
10 #버튼 핀의 입력설정, PULL DOWN 설정
11 #GPIO 15번핀, 입력을 받고, 버튼은 눌렀을때
12 GPIO.setup(button_pin, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
13
14 while 1: #무한반복하면서
15     #만약 버튼핀에 High(1) 신호가 들어오면, "Button pushed!"를 출력
16     if GPIO.input(button_pin) == GPIO.HIGH: #HIGH is 3.3V
17         print("Button pushed!")
18     time.sleep(0.1) #0.1초 딜레이(0.1초 간격으로 버튼 체크)
```

Button pushed!
Button pushed!
Button pushed!
Button pushed!
Button pushed!

1. GPIO 제어 및 테스트

4. 푸시 버튼 스위치 실습 (Event 방식)

1) Event방식

: 일정한 주기로 매번 확인하는 것이 아닌 이벤트가 발생하면 관련 함수를 수행하는 방식

```
1  #필요한 모듈(라이브러리)을 불러옵니다.
2  import RPi.GPIO as GPIO
3  import time
4
5  #button_callback함수를 정의
6  def button_callback(channel):
7      print("Button pushed!")
8
9  #GPIO 초기 세팅
10 GPIO.setmode(GPIO.BCM)
11 GPIO.setwarnings(False)
12 button_pin = 15
13
14 #버튼 핀의 IN/OUT 설정, PULL DOWN 설정
15 #GPIO 15번핀, 입력을 받고, 버튼은 눌렀을때
16 GPIO.setup(button_pin, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
17
18 #Event방식으로 핀의 Rising신호(입력값이 0 -> 1)를 감지하면 button_callback 함수를 실행
19 #GPIO.FALLING (입력값이 1 -> 0)
20 GPIO.add_event_detect(button_pin, GPIO.RISING, callback=button_callback)
21
```

1. GPIO 제어 및 테스트

4. 푸시 버튼 스위치 실습 (Event 방식)

1) Event방식

: 일정한 주기로 매번 확인하는 것이 아닌 이벤트가 발생하면 관련 함수를 수행하는 방식

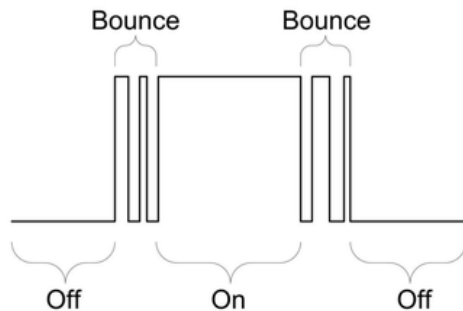
```
21  
22 while 1:  
23     time.sleep(0.1)
```

: 이벤트가 한 번 발생했기 때문에 'Button pushed!' 메시지는 한 번만 발생

```
Button pushed!  
█
```

1. GPIO 제어 및 테스트

3) 스위치의 Bouncing 현상



: 스위치를 누르거나 떼때, 깨끗하게 붙거나 떨어지지 않고, 고속으로 여러 번 on/off가 반복되는 현상

: `bouncetime`을 설정하여 잘못된 신호 입력 방지

`GPIO.add_event_detect( ~ , ~ , ~ , bouncetime=300)`

```
GPIO.add_event_detect(button_pin, GPIO.RISING, callback=button_callback, bouncetime=300)
```


1. GPIO 제어 및 테스트

4. 푸시 버튼 스위치 실습 (Event 방식)

2) 실행파일을 터미널에서 실행하기

: py파일이 있는 디렉토리에서 python 명령어로 실행

```
pi@raspberrypi: ~/webapps/ch05
파일(F) 편집(E) 탭(T) 도움말(H)
pi@raspberrypi:~ $ cd /home/pi/webapps/ch05
pi@raspberrypi:~/webapps/ch05 $ python Btn_Event.py
Button pushed!
Button pushed!
Button pushed!
Button pushed!
Button pushed!
Button pushed!
Button pushed!
```

: python 실행파일 버전 변경하기

\$ python -V

(파이썬 버전을 확인)

\$ sudo ln -f /usr/bin/python3.9 /usr/bin/python (파이썬 실행파일 링크를 변경)

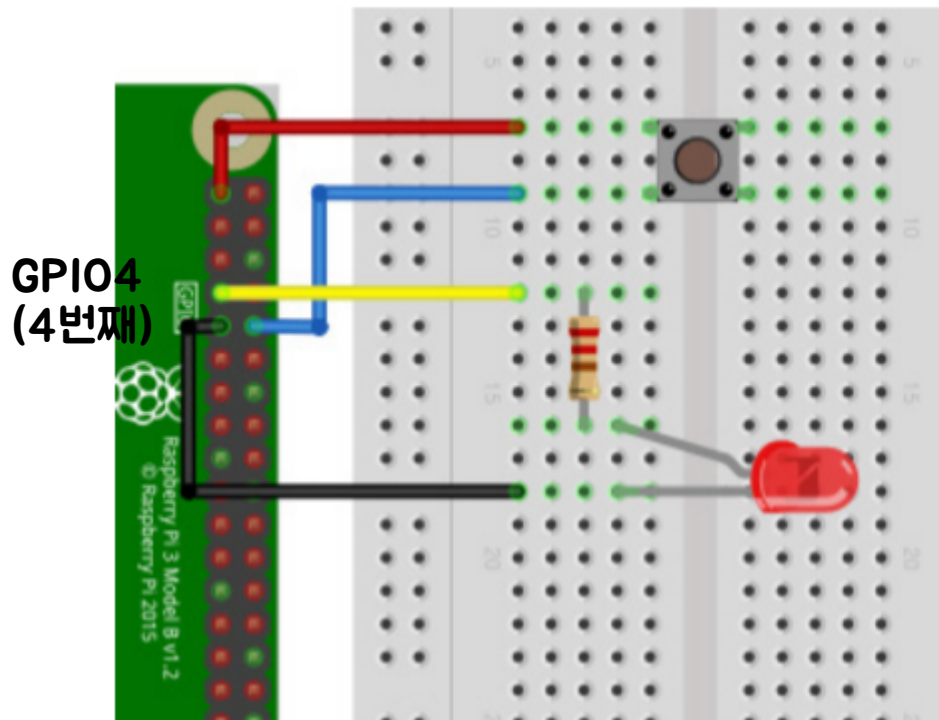
\$ python -V

(파이썬 버전을 다시 확인)

1. GPIO 제어 및 테스트

5. 푸시 버튼 입력으로 LED On/Off 실습

1) 브레드보드 연결하기 (저항 220Ω, LED, Tact스위치)



RP 4	LED
GPIO 15	스위치 연결
VCC(3.3V)	스위치 연결 (제일 마지막에 연결)
GPIO 4	저항
GND	LED -

1. GPIO 제어 및 테스트

5. 푸시 버튼 입력으로 LED On/Off 실습

2) 스위치를 눌렀을 때 LED를 켜고 끄는 함수

```
#button_callback 함수를 정의합니다.
def button_callback(channel):
    global light_on                # Global 변수선언 (LED상태값 저장)
    if light_on == False:         # LED가 꺼져있으면
        GPIO.output(led_pin,1)   # LED ON
        print("LED ON!")
        light_on = True
    else:                          # LED가 켜져있으면
        GPIO.output(led_pin,0)   # LED OFF
        print("LED OFF!")
        light_on = False
```

1. GPIO 제어 및 테스트

5. 푸시 버튼 입력으로 LED On/Off 실습

4) 파이썬 코드 작성하기

```
1  #필요한 모듈(라이브러리)을 불러옵니다.
2  import RPi.GPIO as GPIO
3  import time
4
5  #button_callback 함수를 정의합니다.
6  def button_callback(channel):
7      global light_on          # Global 변수선언 (LED상태값 저장)
8      if light_on == False:    # LED가 꺼져있으면
9          GPIO.output(led_pin,1) # LED ON
10         print("LED ON!")
11         light_on = True
12     else:                    # LED가 켜져있으면
13         GPIO.output(led_pin,0) # LED OFF
14         print("LED OFF!")
15         light_on = False
16
17  #GPIO 초기 세팅
18  GPIO.setmode(GPIO.BCM)
19  GPIO.setwarnings(False)
20  button_pin = 15 #button을 control할 GPIO 번호
21  led_pin = 4     #LED를 control할 GPIO 번호
22  light_on = False #LED 상태를 확인하는 변수
```

1. GPIO 제어 및 테스트

5. 푸시 버튼 입력으로 LED On/Off 실습

4) 파이썬 코드 작성하기

```
23
24 #버튼 핀의 IN 설정, PULL DOWN 설정
25 #GPIO 15번핀, 입력을 받고, 버튼은 눌렀을때
26 GPIO.setup(button_pin, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
27 #GPIO 4번핀, 출력 설정
28 GPIO.setup(led_pin, GPIO.OUT)
29
30 #Event방식으로 핀의 Rising신호(입력값이 0 -> 1)를 감지하면 button_callback 함수를 실행
31 #3000ms 바운스 타임을 설정하여 잘못된 신호를 방지
32 #GPIO.FALLING (입력값이 1 -> 0)
33 GPIO.add_event_detect(button_pin, GPIO.RISING, callback=button_callback, bouncetime=300)
34
35 while 1:
36     time.sleep(0.1)
37
```

LED ON!
LED OFF!
LED ON!
LED OFF!
■

1. GPIO 제어 및 테스트

6.푸시 버튼 입력으로 LED 신호등 실습

```
import RPi.GPIO as GPIO
import time

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)

button_pin = 15
GPIO.setup(button_pin, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

LED_list = [18, 23, 24]
GPIO.setup(LED_list, GPIO.OUT)
GPIO.output(LED_list, False)
```

1. GPIO 제어 및 테스트

6. 푸시 버튼 입력으로 LED 신호등 실습

```
def trafficLight(channel):  
    for i in range(1, 101):  
        num = (i-1) % 3  
  
        GPIO.output(LED_list[num], True)  
        time.sleep(0.5)  
        GPIO.output(LED_list[num], False)
```

```
GPIO.add_event_detect(button_pin, GPIO.RISING, callback=trafficLight, bouncetime=300)
```

```
while 1:  
    time.sleep(0.1) # 0.1초 딜레이
```

1. GPIO 제어 및 테스트

7. 푸시 버튼 입력으로 LED 369게임 실습

```
def LED369(channel):  
    for i in range(1, 101):  
        num = (i-1) % 3  
        num_str = str(i)  
        flag_369 = 0  
  
        if(num_str.find('3') != -1 or num_str.find('6') != -1 or num_str.find('9') != -1):  
            print("x(%s)" % num_str)  
            flag_369 = 1  
        else:  
            print(num_str)  
  
        if(flag_369==1):  
            GPIO.output(LED_list[num], False)  
        else:  
            GPIO.output(LED_list[num], True)  
  
        time.sleep(0.5)  
        GPIO.output(LED_list[num], False)
```

```
GPIO.add_event_detect(button_pin,GPIO.RISING,callback=LED369, bouncetime=300)
```


1. GPIO 제어 및 테스트

Today 미션

369 LED



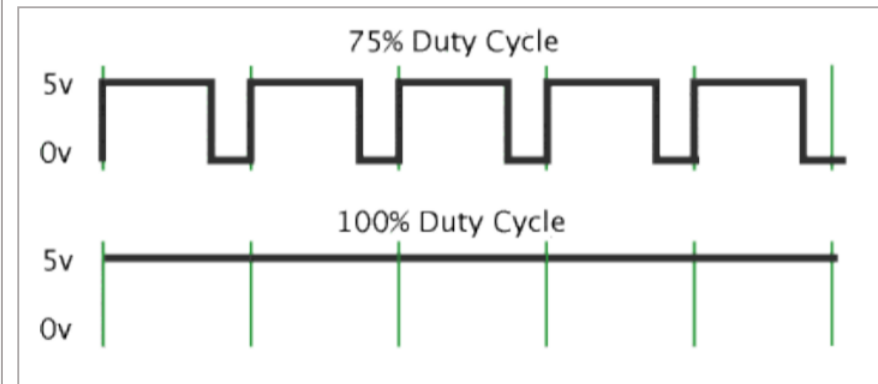
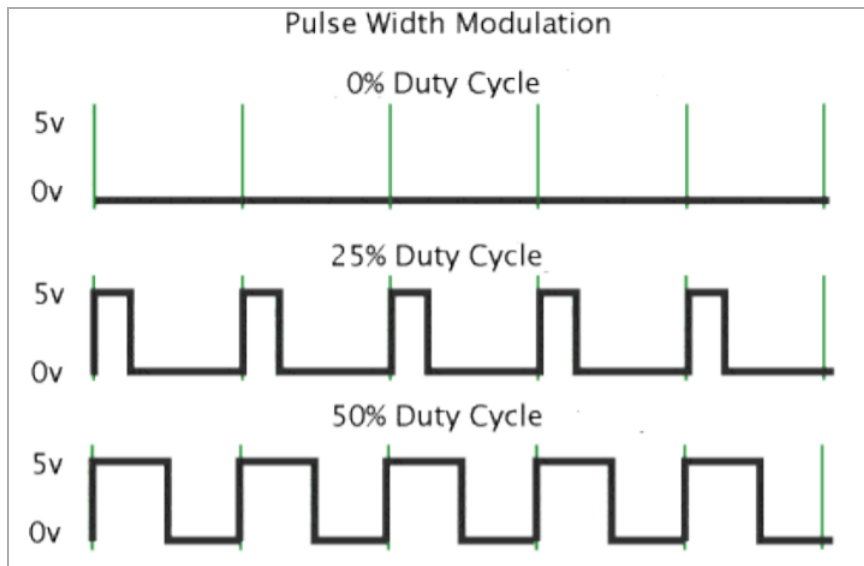
OLED
369일때 x 표시

부저
X 일때 이펙트 소리내기

2. PWM 제어

1. PWM (Pluse Width Modulation)

- 1) 주기적으로 반복되는 On, Off신호를 통해서 모터, 팬의 속도나 LED의 밝기 등을 제어
- 2) 반복되는 Pulse신호의 주기에서 On신호가 지속되는 시간의 비율을 Duty Cycle라고 한다.
- 3) Duty Cycle (듀티비)가 클수록 팬속도나 LED의 밝기가 커짐



2. PWM 제어

2. PWM으로 LED 실습

1) PWM 사용하기

: PWM 인스턴스 p를 생성. 매개변수 (핀번호, 주파수)

`p = GPIO.PWM(channel, frequency)`

: PWM 시작

`p.start(dc)` (듀티비 dc는 0에서 100사이의 값)

: 주파수 설정

`p.ChangeFrequency(freq)` (주파수(Hz) 변경)

: 듀티비 변경

`p.ChangeDutyCycle(dc)` ($0.0 \leq dc \leq 100.0$)

: PWM 종료

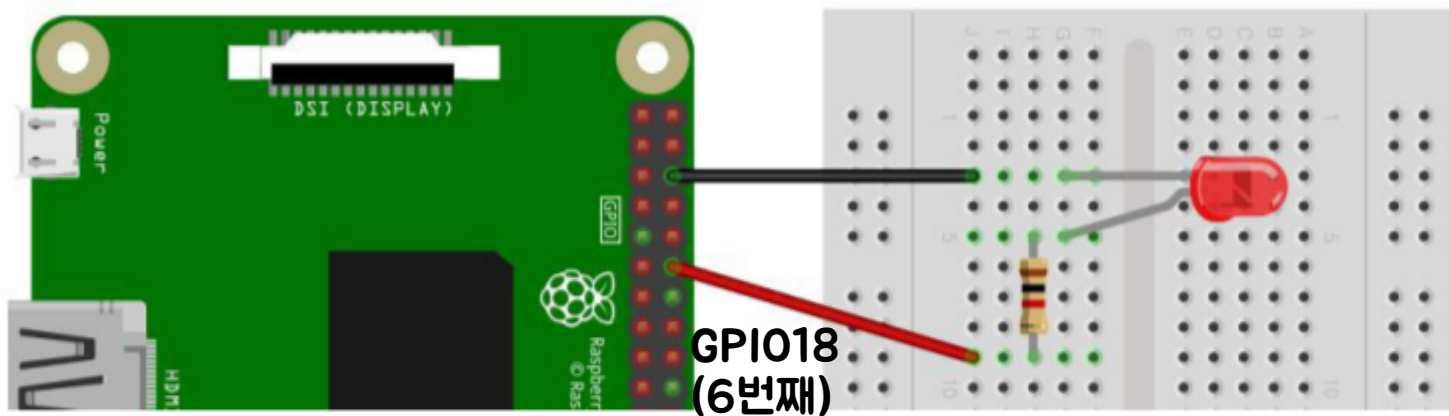
`p.stop()`

2. PWM 제어

2. PWM으로 LED 실습

2) 브레드보드 연결하기 (저항 220Ω, LED)

RP 4	LED
GPIO 18	+
GND	-



2. PWM 제어

2. PWM으로 LED 실습

3) 파이썬 코드 작성하기 (듀티비, 딜레이 시간을 변화시켜 보기)

```
1 import RPi.GPIO as GPIO
2 import time
3
4 GPIO.setmode(GPIO.BCM)
5 GPIO.setwarnings(False)
6 GPIO.setup(18, GPIO.OUT) #GPIO 18번핀을 출력으로 설정
7
8 #PWM인스턴스 p를 만들고 GPIO 18번을 PWM핀으로 설정, 주파수 = 50Hz
9 p=GPIO.PWM(18, 50)
10
11 #PWM시작, 듀티비 = 0
12 p.start(0)
13
```

2. PWM 제어

2. PWM으로 LED 실습

3) 파이썬 코드 작성하기 (듀티비, 딜레이 시간을 변화시켜 보기)

```
13
14 try:      #예외 처리문
15     while 1:
16         for dc in range(0, 101, 5):
17             p.ChangeDutyCycle(dc) #듀티비를 0~100까지 5씩 증가
18             time.sleep(0.3)      #0.3초 딜레이
19         for dc in range(100, -1, -5):
20             p.ChangeDutyCycle(dc) #듀티비를 100~0까지 -5씩 증가
21             time.sleep(0.3)      #0.3초 딜레이
22 except KeyboardInterrupt:
23     pass
24
25 p.stop()      #PWM 종료
26 GPIO.cleanup() #GPIO 설정을 초기화
```

4) 실행 종료

: Ctrl + C

2. PWM 제어

3. PWM으로 부저 실습

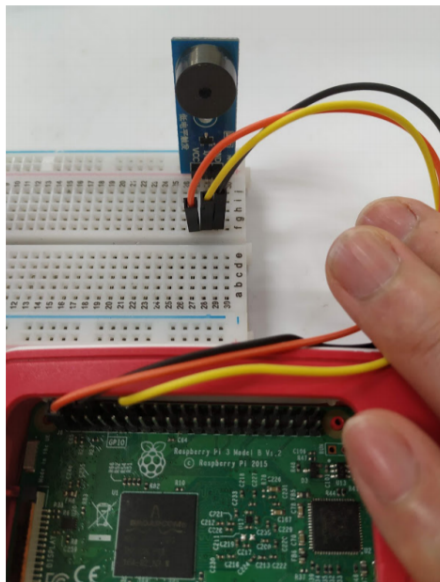
1) 부저의 종류

: Active - LED의 On/Off처럼 일정한 전원만 가해주면 소리가 발생. 단 "삐~"하는 경고음만 가능

: Passive - 전원만 가해서는 소리가 나지 않고 일정한 주파수를 가할 때 소리가 발생. 주파수의 높낮이에 따라 음 조절 가능



2) 브레드보드 연결하기 (부저)



RP 4	부저
GPIO 18	I/O
VCC(3.3V)	VCC
GND	GND

2. PWM 제어

3. PWM으로 부저 실습

3) 음계 주파수

Octave 및 음계별 표준 주파수									
(단위 : Hz)									
음계 \ Octave	1	2	3	4	5	6	7	8	
C(도)	32.7032	65.4064	130.8128	261.6256	523.2511	1046.502	2093.005	4186.009	
C #	34.6478	69.2957	138.5913	277.1826	554.3653	1108.731	2217.461	4434.922	
D(레)	36.7081	73.4162	146.8324	293.6648	587.3295	1174.659	2349.318	4698.636	
D #	38.8909	77.7817	155.5635	311.1270	622.2540	1244.508	2489.016	4978.032	
E(미)	41.2034	82.4069	164.8138	329.6276	659.2551	1318.510	2637.020	5274.041	
F(파)	43.6535	87.3071	174.6141	349.2282	698.4565	1396.913	2793.826	5587.652	
F #	46.2493	92.4986	184.9972	369.9944	739.9888	1479.978	2959.955	5919.911	
G(솔)	48.9994	97.9989	195.9977	391.9954	783.9909	1567.982	3135.963	6271.927	
G #	51.9130	103.8262	207.6523	415.3047	830.6094	1661.219	3322.438	6644.875	
A(라)	55.0000	110.0000	220.0000	440.0000	880.0000	1760.000	3520.000	7040.000	
A #	58.2705	116.5409	233.0819	466.1638	932.3275	1864.655	3729.310	7458.620	
B(시)	61.7354	123.4708	246.9417	493.8833	987.7666	1975.533	3951.066	7902.133	

2. PWM 제어

3. PWM으로 부저 실습

4) 파이썬 코드 작성하기

```
1  import RPi.GPIO as GPIO
2  import time
3
4  GPIO.setmode(GPIO.BCM)
5  GPIO.setwarnings(False)
6  GPIO.setup(18, GPIO.OUT)
7
8  #PWM인스턴스 P를 만들고 GPIO 18번을 PWM핀으로 설정, 주파수 = 1Hz
9  p=GPIO.PWM(18,1)
10
11 #주파수 변경을 위한 리스트 변수 선언
12 Frq = [262, 294, 330, 349, 392, 440, 493, 523]
13
14 #듀티비 50로 시작
15 p.start(50)
16
```

2. PWM 제어

3. PWM으로 부저 실습

4) 파이썬 코드 작성하기

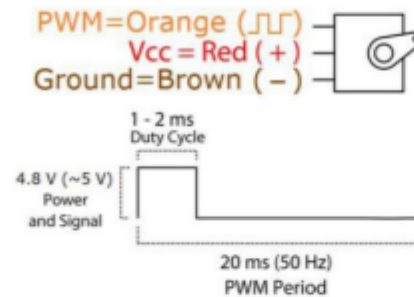
```
16
17 try:      #예외 처리문
18     while 1:
19         for fr in Frq:
20             p.ChangeFrequency(fr) #주파수 변경(소리변경)
21             time.sleep(0.5)      #0.5초 딜레이
22 except KeyboardInterrupt:
23     pass
24
25 p.stop()      #PWM 종료
26 GPIO.cleanup() #GPIO 설정을 초기화
```

2. PWM 제어

4. PWM으로 서보모터 실습

1) 서보모터의 이해

: 전원이 연결된 상태에서 듀티비 값을 따로 입력시켜주면 입력 값에 맞춰 각도가 변함

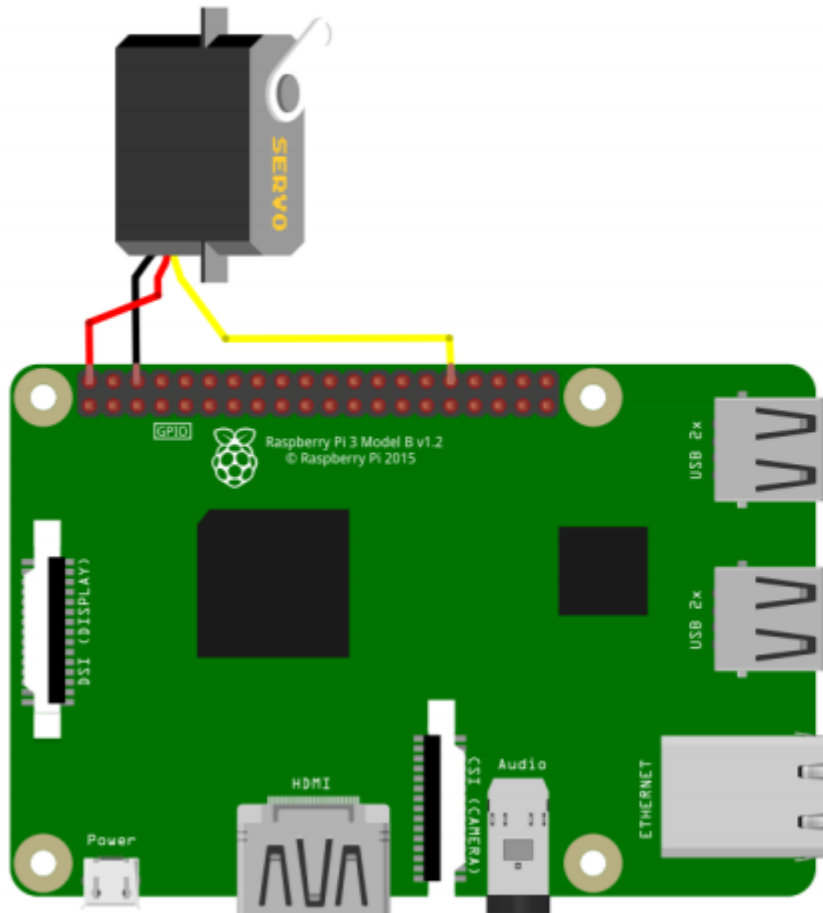


각도	듀티비
0	2.5
90	7.5
180	12.5

2. PWM 제어

4. PWM으로 서보모터 실습

2) 브레드보드 연결하기 (서보모터 SG-90)



RP 4	서보모터
GPIO 18	오렌지색
VCC	빨간색
GND	갈색

2. PWM 제어

4. PWM으로 서보모터 실습

3) 파이썬 코드 작성하기

```
1  import RPi.GPIO as GPIO
2  import time
3
4
5  GPIO.setmode(GPIO.BCM)
6  GPIO.setwarnings(False)
7  SERVO_PIN=18
8  GPIO.setup(SERVO_PIN, GPIO.OUT)
9
10 #PWM인스턴스 P를 만들고 GPIO 18번을 PWM핀으로 설정, 주파수 = 50Hz
11 p = GPIO.PWM(SERVO_PIN, 50)
12
13 #듀티비 0으로 시작
14 p.start(0)
15
```

2. PWM 제어

4. PWM으로 서보모터 실습

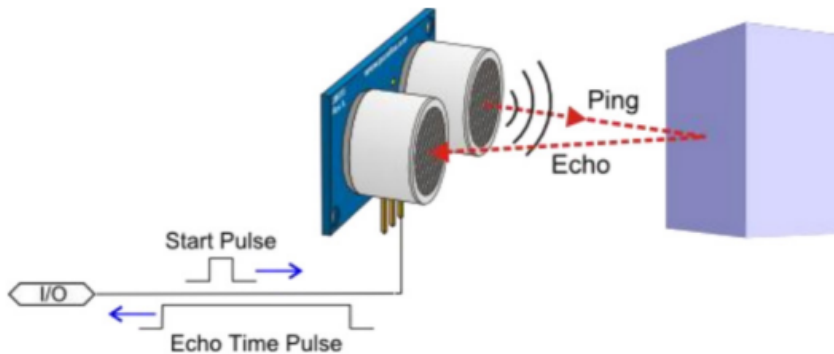
3) 파이썬 코드 작성하기

```
16 try:      #예외 처리문
17     while True:
18         # 약 2.5 ~ 12.5 의 범위값을 넘어가면 작동하지 않음
19         p.ChangeDutyCycle(2.5) # 0도
20         time.sleep(1)
21         p.ChangeDutyCycle(3.5)
22         time.sleep(1)
23         p.ChangeDutyCycle(4.5)
24         time.sleep(1)
25         p.ChangeDutyCycle(5.5)
26         time.sleep(1)
27         p.ChangeDutyCycle(6.5)
28         time.sleep(1)
29         p.ChangeDutyCycle(7.5) # 90도
30         time.sleep(1)
31         p.ChangeDutyCycle(8.5)
32         time.sleep(1)
33         p.ChangeDutyCycle(9.5)
34         time.sleep(1)
35         p.ChangeDutyCycle(10.5)
36         time.sleep(1)
37         p.ChangeDutyCycle(11.5)
38         time.sleep(1)
39         p.ChangeDutyCycle(12.5) # 180도
40         time.sleep(1)
41 except KeyboardInterrupt:
42     p.stop()      #PWM 종료
43     GPIO.cleanup() #GPIO 설정을 초기화
```

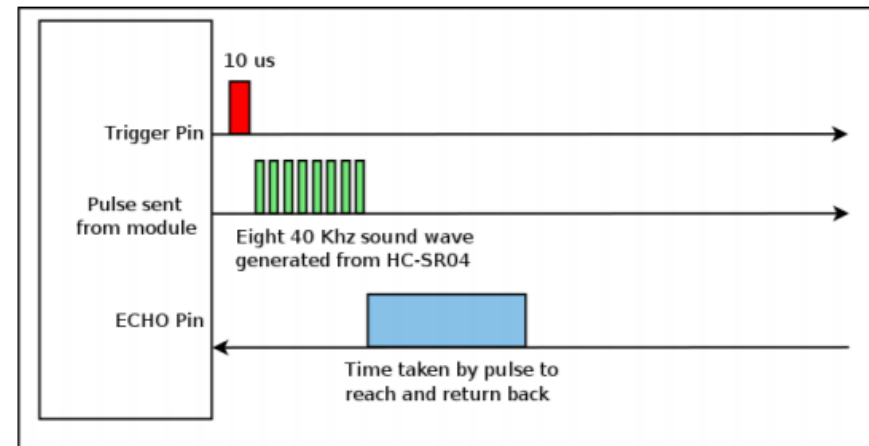
3. 초음파센서(HC-SR04)

1. 초음파센서 (HC-SR04) 실습

1) 초음파센서의 원리



▲ HC-SR04 동작원리



▲ HC-SR04 동작타이밍

: Trigger Pin에 스타트 펄스신호를 주면 HC-SR04에서 사운드웨이브를 전방으로 쏘고 물체에 반사되어 돌아오는 시간을 Echo핀으로 보냄

2) 거리 구하는 공식

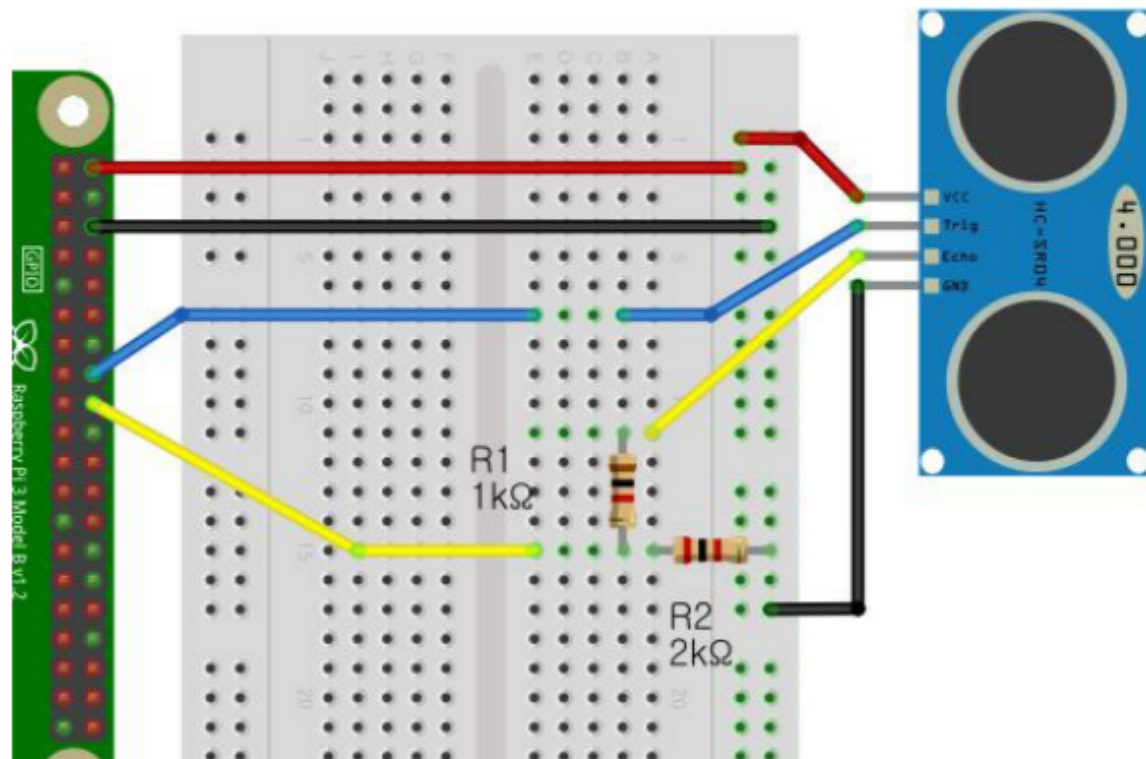
: 물체와의 거리 = 소리의 초당 이동거리 X (걸린시간/2)

: 예 - 물체와의 거리 0.1m = 약343m/s X (588 μ s / 2)

3. 초음파센서(HC-SR04)

1. 초음파센서 (HC-SR04) 실습

3) 브레드보드 연결하기 (HC-SR04, 저항 2K Ω , 1K Ω)



RP 4	초음파센서
VCC(5V)	VCC
GPIO 23	Trigger 핀
GPIO 24	Echo 핀
GND	GND

3. 초음파센서(HC-SR04)

1. 초음파센서 (HC-SR04) 실습

4) 전압 분배 회로

Electric Parameter

Working Voltage	DC 5 V
Working Current	15mA
Working Frequency	40Hz
Max Range	4m
Min Range	2cm
MeasuringAngle	15 degree
Trigger Input Signal	10uS TTL pulse
Echo Output Signal	Input TTL lever signal and the range in proportion
Dimension	45*20*15mm

3. 초음파센서(HC-SR04)

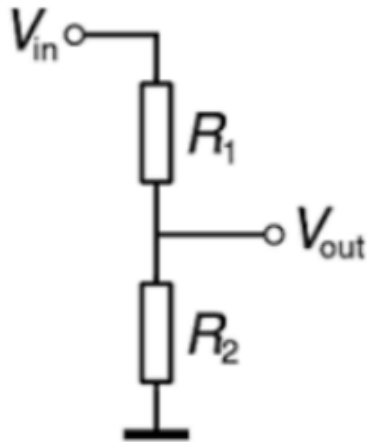
1. 초음파센서 (HC-SR04) 실습

4) 전압 분배 회로

5V를 3.3V로 낮추는 방법 (참고)

라즈베리 파이의 GPIO 핀은 3.3V 이상 입력하면 안되므로 5V로 동작하는 HC-SR04에서 나오는 Echo 핀의 신호를 5V→3.3V로 전압을 낮추어 입력을 받아야 합니다.

그러므로 아래의 회로와 같이 5V의 전압을 전압분배로 낮추는 회로를 추가 합니다.



▲ Echo 핀 전압분배

위의 그림에서 $V_{in} = 5V$, $R_1 = 1K$, $R_2 = 2K$ 라 할 때

$V_{out} = V_{in} \times R_2 / (R_1 + R_2) = 5 \times 2 / 3 = 3.33 V$ 입니다.

여기서 V_{in} 은 Echo 핀이고 V_{out} 은 라즈베리 파이의 입력 핀이 됩니다.

3. 초음파센서(HC-SR04)

1. 초음파센서 (HC-SR04) 실습

5) 파이썬 코드 작성하기

```
1 import RPi.GPIO as GPIO
2 import time
3
4 GPIO.setmode(GPIO.BCM)
5 GPIO.setwarnings(False)
6 #Trig와 Echo핀 번호 설정(BCM방식)
7 TRIG = 23
8 ECHO = 24
9 #Trig와 Echo의 입/출력 설정
10 print("Distance measurement in progress")
11 GPIO.setup(TRIG, GPIO.OUT)
12 GPIO.setup(ECHO, GPIO.IN)
13
14
15 #Trig핀의 신호를 0으로 출력
16 print("Waiting for sensor to settle")
17 GPIO.output(TRIG, False)
18 time.sleep(2)
19
```

3. 초음파센서(HC-SR04)

1. 초음파센서 (HC-SR04) 실습

5) 파이썬 코드 작성하기

```
20 try:
21     while True:
22         #초음파 센서에 트리거 신호를 발생
23         GPIO.output(TRIG, True)
24         time.sleep(0.00001)          # 10us
25         GPIO.output(TRIG, False)
26
27         #ECHO신호 측정
28         while GPIO.input(ECHO)==0: #ECHO핀 상승시간
29             start = time.time()
30         while GPIO.input(ECHO)==1: #ECHO핀 하강시간
31             stop = time.time()
32
33         #거리계산
34         check_time = stop-start
35         #초당 34300cm이동, 돌아오는 시간을 계산하여 /2를 함
36         distance = check_time * 34300/2
37         print("Distance : %.1f cm" % distance)
38         time.sleep(0.4)              #0.4초 간격으로 센서 측정
39 except KeyboardInterrupt:
40     print("measurement stopped by User")
41     GPIO.cleanup()
```

초음파센서(HC-SR04)

가장 먼 거리

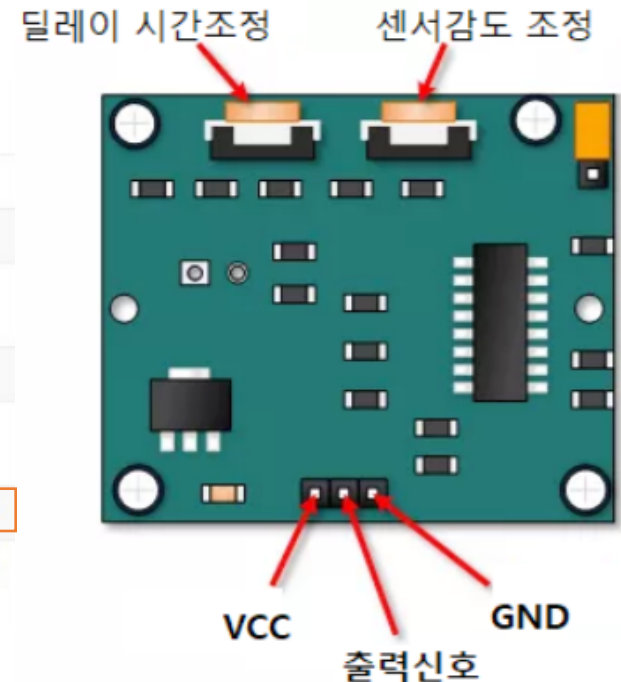
4. 적외선 감지센서(HC-SR501)

1. 적외선 감지센서(PIR, HC-SR501) 실습

1) 초음파센서의 원리



Weight	6 g
Dimensions	32 x 24 x 35 mm
OPERATING VOLTAGE	4.5~20 Volt.
LENS ANGLE	<100 ° cone angle
QUIESCENT CURRENT	<50uA
LEVEL OUTPUT	High 3.3 V /Low 0V
DELAY TIME	5-200S(adjustable)

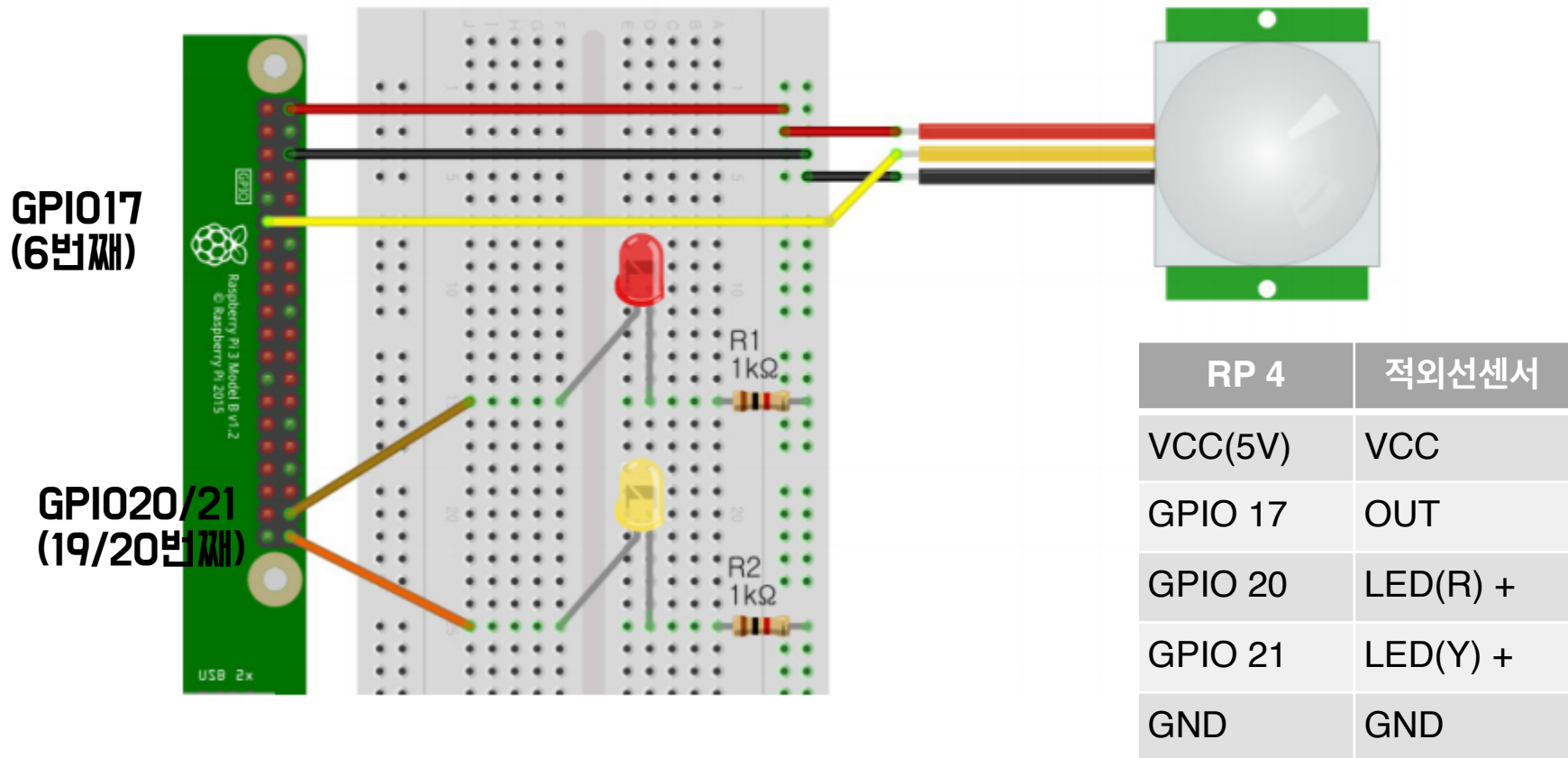


- : 적외선을 통해 사람의 움직임을 감지하는 센서
- : 건물의 복도나 현관문 천장에서 사람의 움직임을 감지 되었을 때 자동으로 조명을 켜주는데 이용되는 센서
- : 적외선의 변화가 있을 시 High(1)신호를 출력, 없을 시 Low(0)신호를 출력

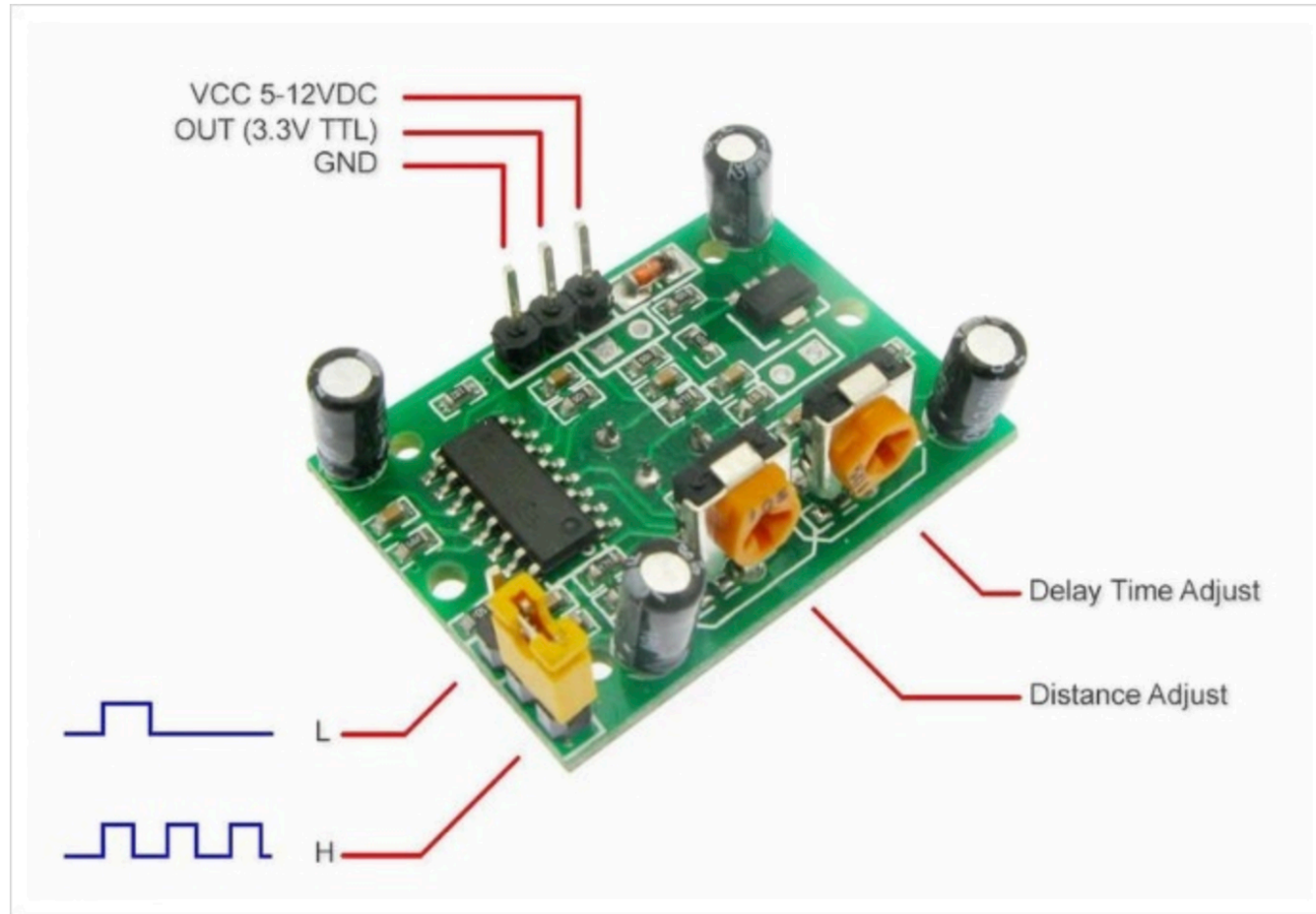
4. 적외선 감지센서(HC-SR501)

1. 적외선 감지센서(PIR, HC-SR501) 실습

2) 브레드보드 연결하기 (적외선 감지센서 HC-SR501, Yello-LED, Red-LED, 저항 1K Ω 2개)



4. 적외선 감지센서(HC-SR501)



4. 적외선 감지센서(HC-SR501)

1. 적외선 감지센서(PIR, HC-SR501) 실습

3) 파이썬 코드 작성하기

```
1 import RPi.GPIO as GPIO
2 import time
3
4 GPIO.setmode(GPIO.BCM)
5 GPIO.setwarnings(False)
6 #LED, 센서 핀 번호 설정(BCM방식)
7 led_R=20
8 led_Y=21
9 sensor=17
10 #LED, 센서 입/출력 설정
11 GPIO.setup(led_R, GPIO.OUT)
12 GPIO.setup(led_Y, GPIO.OUT)
13 GPIO.setup(sensor, GPIO.IN)
14
15 #PIR센서 준비
16 print("PIR Ready.....")
17 time.sleep(2)
18
```

```
19 try:
20     while True:
21         #센서가 High(1), 인식 할때
22         if GPIO.input(sensor) ==1:
23             GPIO.output(led_Y, 1)
24             GPIO.output(led_R, 0)
25             print("Motion Detected !")
26             time.sleep(0.1)
27         #센서가 Low(0), 인식 안할때
28         if GPIO.input(sensor) ==0:
29             GPIO.output(led_Y, 0)
30             GPIO.output(led_R, 1)
31             time.sleep(0.1)
32 except KeyboardInterrupt:
33     print("Stopped by User")
34     GPIO.cleanup()
```

5. 아날로그 신호와 SPI 통신

1. 아날로그 신호

- 1) 아날로그(analogue)는 어떤 수치를 '길이', '각도' 또는 '전압' 과 같이 연속적으로 변하는 물리량으로 나타내는 것.
- 2) 바늘의 각도로 표시해주는 속도측정계, 수은주의 길이로 나타내는 온도계, 깊이가 다른 홈과 바늘의 마찰로 소리를 나타내는 LP음반

2. 디지털 신호

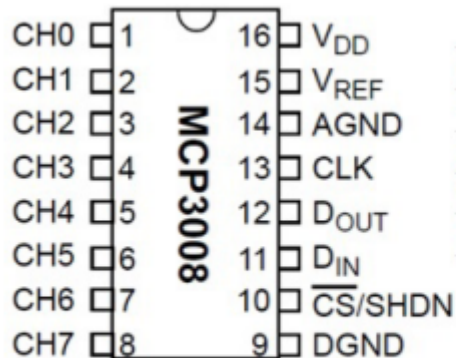
- 1) 데이터를 일련의 이산 값들로 표현하기 위해 사용되는 신호
- 2) 일반적으로 On/Off, High/Low, 1/0으로 표현할 수 있다.

5. 아날로그 신호와 SPI 통신

3. 아날로그 신호를 라즈베리 파이로 읽어오기

- 1) 아날로그 신호를 디지털로 변환하여 읽어 올 때 ADC(Analog to Digital Converter) 기능을 가진 MCP3008를 사용

: MCP3008 데이터 시트



해상도:	10 bit
채널 수:	8 Channel
샘플링 속도:	200 kS/s
입력 타입:	Single-Ended
인터페이스 타입:	Serial, 4-Wire, SPI

: 8개의 입력 채널

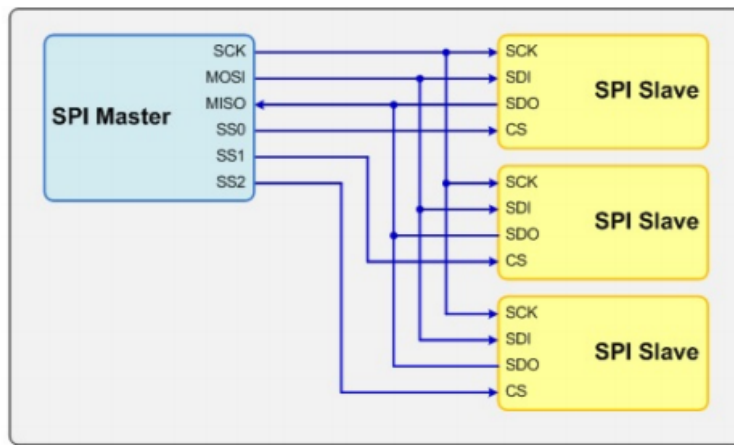
: 해상도 10bit - 아날로그 전압을 10bit만큼 분해 가능 ($2^{10}=1024$)

아날로그 최대입력 전압이 3.3V라면, $3.3V / 1024 = 3.22mV$ 단위로 쪼개어 표현 가능
(0000000001) = 3.22mV

5. 아날로그 신호와 SPI 통신

4. SPI 통신의 이해

- 1) SPI(Serial Peripheral Interface) 통신은 라즈베리파이, 아두이노 같은 마스터 장치와 주변장치들간의 시리얼 통신을 위한 통신방식



- 2) SPI 통신을 위해서는 기본적으로 3개의 SPI 신호선 + 슬레이브 개수 만큼의 신호선이 필요함.

: MISO (Master In Slave Out) : 슬레이브에서 마스터로 **데이터 이동**

: MOSI (Master Out Slave In) : 마스터에서 슬레이브로 **데이터 이동**

: SCLK (Serial Clock) : 통신의 동기를 맞추기 위한 신호, 마스터에서 만들어 전송

: CS or SS (Chip Select or Slave Select) : 슬레이브를 선택하는 신호선

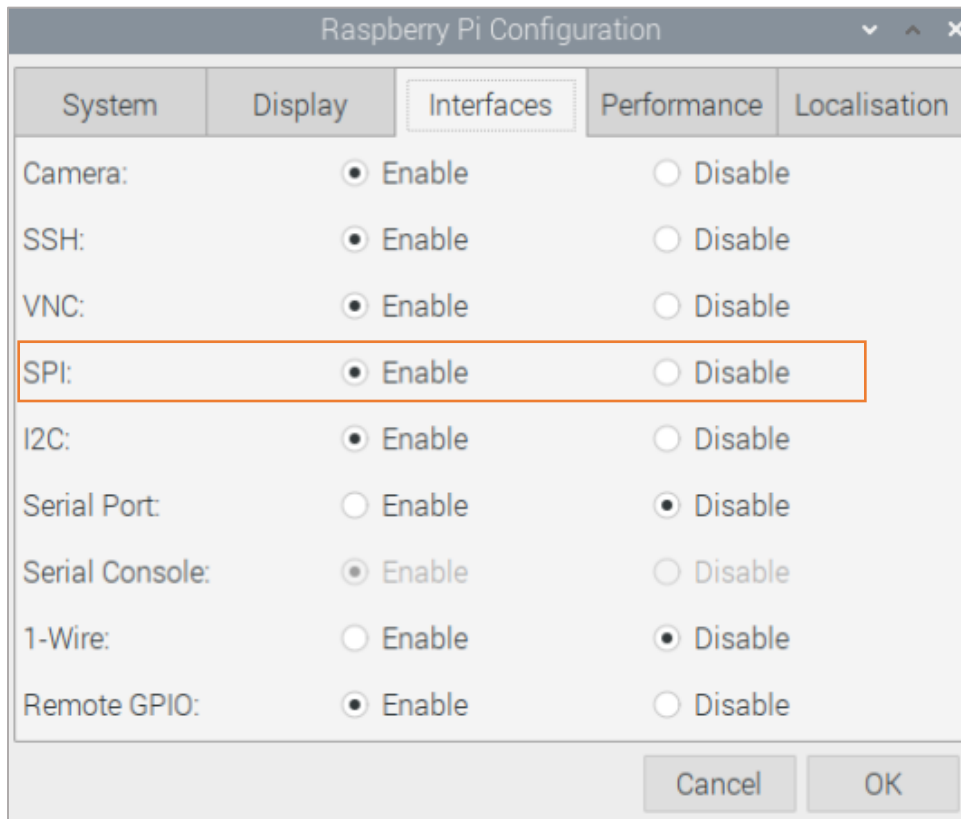
5. 아날로그 신호와 SPI 통신

5. 라즈베리파이 SPI 설정하기

1) 기본설정 - Raspberry Pi Configuration - Interfaces - SPI (Enable)

2) 설정 후 재부팅

: \$ sudo reboot



5. 아날로그 신호와 SPI 통신

6. 파이썬에서 SPI라이브러리 사용하기

1) 파이썬3용 라이브러리 설치

: \$ sudo pip3 install spidev

2) 라이브러리 설치 확인

: \$ pip3 show spidev

or

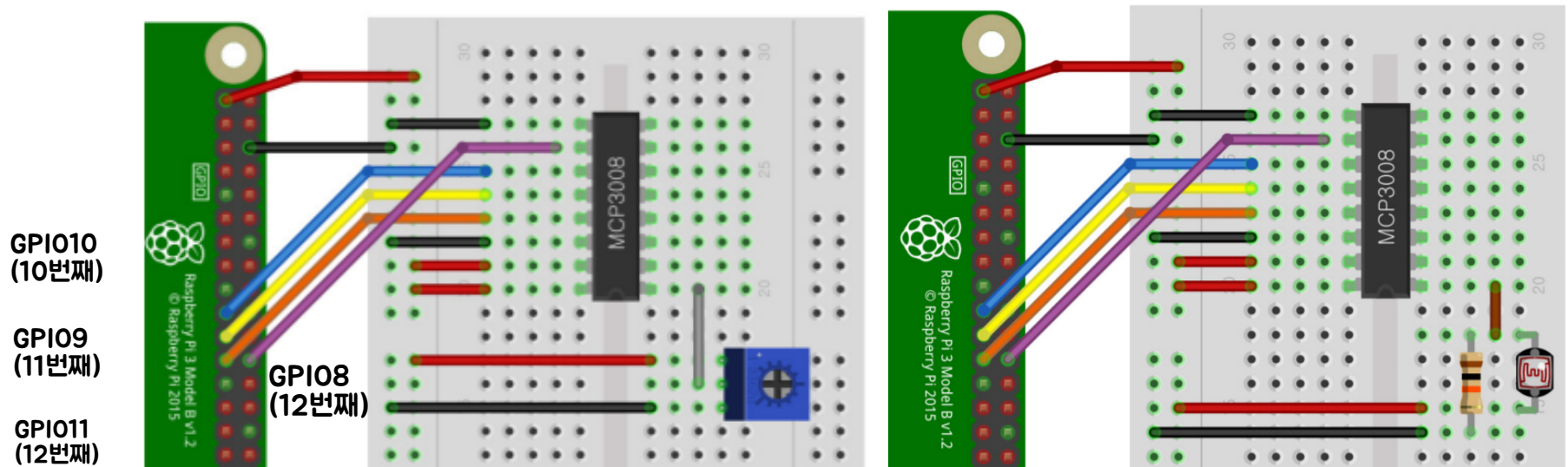
: \$ pip3 list | grep spi

```
pi@raspberrypi:~ $ pip3 show spidev
Name: spidev
Version: 3.4
Summary: Python bindings for Linux SPI access through spidev
Home-page: http://github.com/doceme/py-spidev
Author: Volker Thoms
Author-email: unconnected@gmx.de
License: GPLv2
Location: /home/pi/.local/lib/python3.7/site-packages
Requires:
Required-by: Adafruit-GPIO
```

5. 아날로그 신호와 SPI 통신

7. 아날로그 전압을 MCP3008로 읽어오기

1) 브레드보드 연결하기 (MCP3008, 10K Ω 가변저항, 저항 10K Ω , 조도센서)



2) 조도센서 (LDR)

: 광센서 모듈로써 빛의 세기에 따라서 저항값이 달라지는 소자

: 주위가 밝으면 저항값이 작아지고, 주위가 어두우면 저항값이 커짐

RP 4	MCP3008
VCC(3.3V)	VCC
GPIO11(CLK)	CLK
GPIO9(MISO)	DOUT
GPIO10(MOSI)	DIN
GPIO 8 (CS)	CS
GND	GND

5. 아날로그 신호와 SPI 통신

7. 아날로그 전압을 MCP3008로 읽어오기

3) 파이썬 코드 작성하기

```
1  import spidev #spi라이브러리를 불러온다.
2  import time
3
4  #SPI 인스턴스 spi생성
5  spi = spidev.SpiDev()
6  #라즈베리와 SPI통신 시작하기
7  spi.open(0,0)    #open(bus, device)
8  #SPI 통신 속도 설정
9  spi.max_speed_hz = 100000
10
11 #딜레이 시간(센서 측정 간격)
12 delay = 0.5
13 #MCP3008 채널 중 센서에 연결한 채널 설정
14 pot_channel = 0
15
```


5. 아날로그 신호와 SPI 통신

7. 아날로그 전압을 MCP3008로 읽어오기

3) 파이썬 코드 작성하기

```
15
16 # 0-7까지 8개의 채널에서 SPI데이터를 읽어 옵니다.
17 def readadc(adcnun):
18     if adcnun > 7 or adcnun < 0:
19         return -1
20     r = spi.xfer2([1, (8+adcnun)<<4, 0])
21     data = ((r[1]&3) << 8) + r[2]
22     return data
23
24 while True:
25     #readadc 함수로 pot_channel의 SPI데이터를 읽어옵니다.
26     value = readadc(pot_channel)
27     real_voltage = value*3.3/1024
28     print("-----")
29     print("Value: %d Real Volte : %f" % (value, real_voltage))
30     time.sleep(delay) #delay시간만큼 기다립니다.
```

Value: 0	Real Volte : 0.000000
Value: 0	Real Volte : 0.000000
Value: 0	Real Volte : 0.000000
Value: 0	Real Volte : 0.000000

5. 아날로그 신호와 SPI 통신

8. 조이스틱 실습

1) 조이스틱은 X축, Y축의 움직임에 따라 저항값이 변하는 가변저항이 달려 있음

: 스틱이 움직이지 않을 때 중간값 약 500

10bit 안에서 값이 변함 (0~1023)

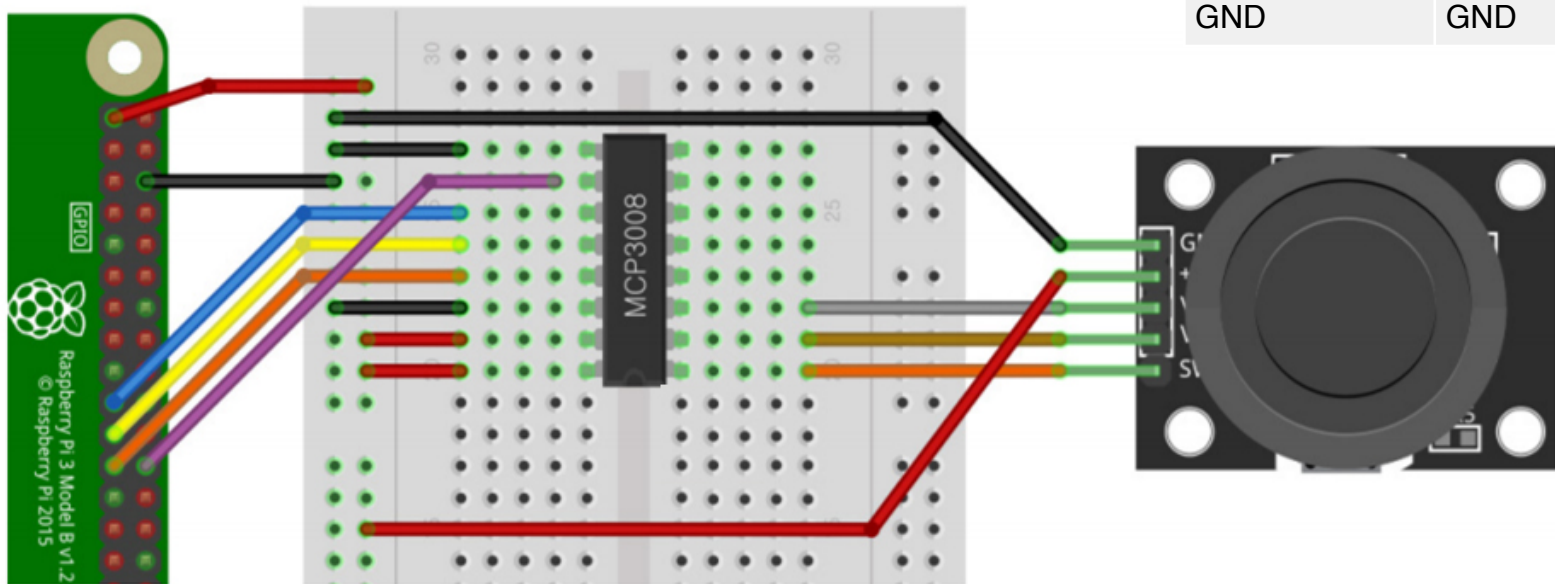
스위치의 경우 눌르면 0



5. 아날로그 신호와 SPI 통신

8. 조이스틱 실습

2) 브레드보드 연결하기 (MCP3008, 조이스틱 모듈)



RP 4	MCP3008
VCC(3.3V)	VCC
GPIO11(CLK)	CLK
GPIO9(MISO)	DOUT
GPIO10(MOSI)	DIN
GPIO 8 (CS)	CS
GND	GND

5. 아날로그 신호와 SPI 통신

8. 조이스틱 실습

3) 파이썬 코드 작성하기

```
1  import spidev #spi라이브러리를 불러온다.
2  import time
3
4  #SPI 인스턴스 spi생성
5  spi = spidev.SpiDev()
6  #라즈베리와 SPI통신 시작하기
7  spi.open(0,0) #open(bus, device)
8  #SPI 통신 속도 설정
9  spi.max_speed_hz = 100000
10
11 #딜레이 시간(센서 측정 간격)
12 delay = 0.5
13 #MCP3008 채널 중 센서에 연결한 채널 설정
14 sw_channel = 0
15 vrx_channel = 1
16 vry_channel = 2
17
```

5. 아날로그 신호와 SPI 통신

8. 조이스틱 실습

3) 파이썬 코드 작성하기

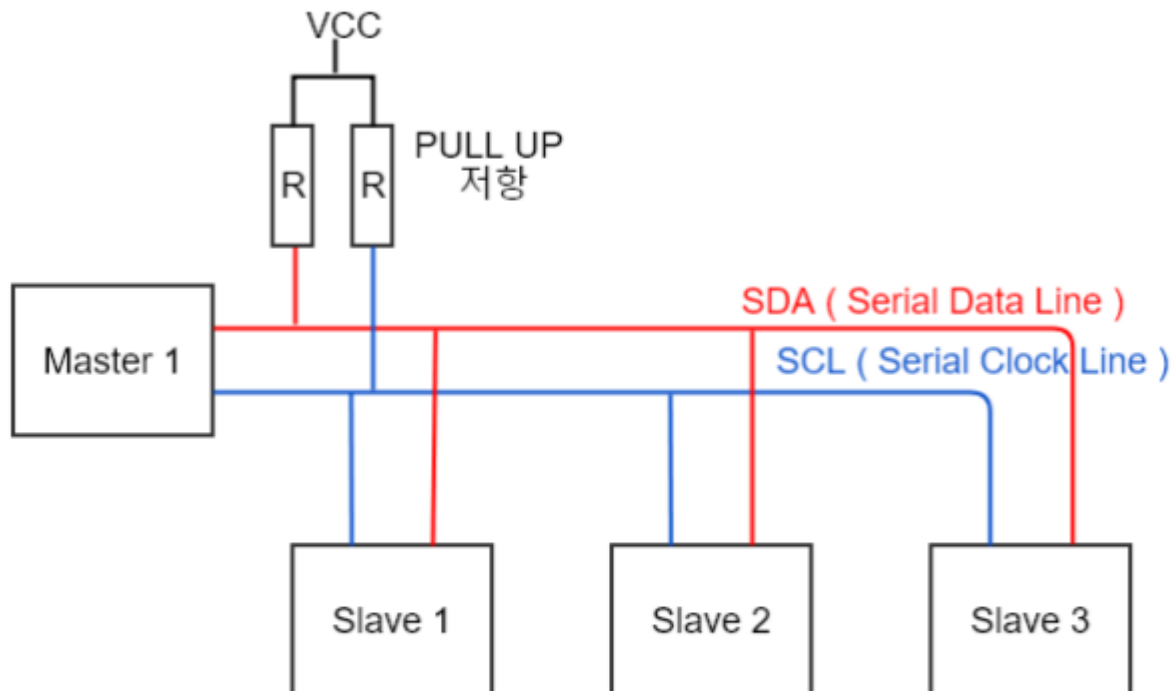
```
17
18 # 0~7까지 8개의 채널에서 SPI데이터를 읽어 옵니다.
19 def readadc(adcnun):
20     if adcnun > 7 or adcnun < 0:
21         return -1
22     r = spi.xfer2([1, (8+adcnun)<<4, 0])
23     data = ((r[1]&3) << 8) + r[2]
24     return data
25
26 while True:
27     #readadc 함수로 각 channel의 SPI데이터를 읽어옵니다.
28     sw_val = readadc(sw_channel) #클릭하면 0
29     vrx_pos = readadc(vrx_channel) #1~1023값 사이를 움직임, 중간값 500정도
30     vry_pos = readadc(vry_channel) #1~1023값 사이를 움직임, 중간값 500정도
31     #출력
32     print("X : %d, Y : %d, SW : %d" % (vrx_pos, vry_pos, sw_val))
33     time.sleep(delay) #delay시간만큼 기다립니다.
```

```
X : 517, Y : 525, SW : 383
X : 517, Y : 524, SW : 250
X : 1, Y : 525, SW : 13
X : 1, Y : 525, SW : 14
X : 29, Y : 525, SW : 83
X : 676, Y : 525, SW : 255
X : 1023, Y : 525, SW : 181
X : 940, Y : 525, SW : 146
X : 516, Y : 525, SW : 3
X : 517, Y : 1023, SW : 56
X : 518, Y : 1023, SW : 47
X : 517, Y : 525, SW : 204
X : 517, Y : 343, SW : 138
X : 516, Y : 1, SW : 70
X : 517, Y : 1, SW : 3
X : 517, Y : 476, SW : 66
X : 517, Y : 525, SW : 99
X : 516, Y : 525, SW : 439
X : 517, Y : 525, SW : 389
X : 516, Y : 525, SW : 115
X : 515, Y : 525, SW : 0
```

6. I²C 통신

1. I²C 통신 개요

- 1) I²C(Inter-Integrated Circuit)는 시리얼 통신
- 2) SDA, SCL 2개의 통신라인만으로 구성 가능
- 3) 슬레이브 기기가 추가 될 때마다 CS(chip select) 통신라인의 추가 없음
- 4) 속도가 느려서 저속의 주변장치 사이의 통신에만 사용



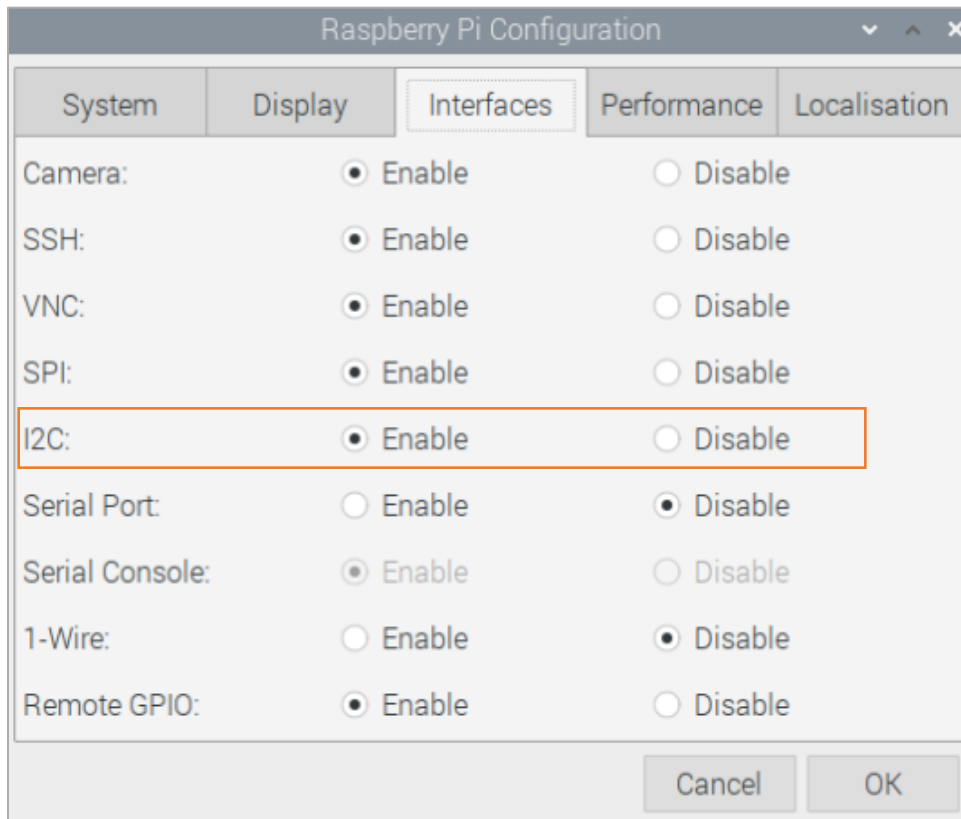
6. I2C 통신

2. 라즈베리파이 I2C 설정하기

1) 기본설정 - Raspberry Pi Configuration - Interfaces - I2C (Enable)

2) 설정 후 재부팅

: \$ sudo reboot



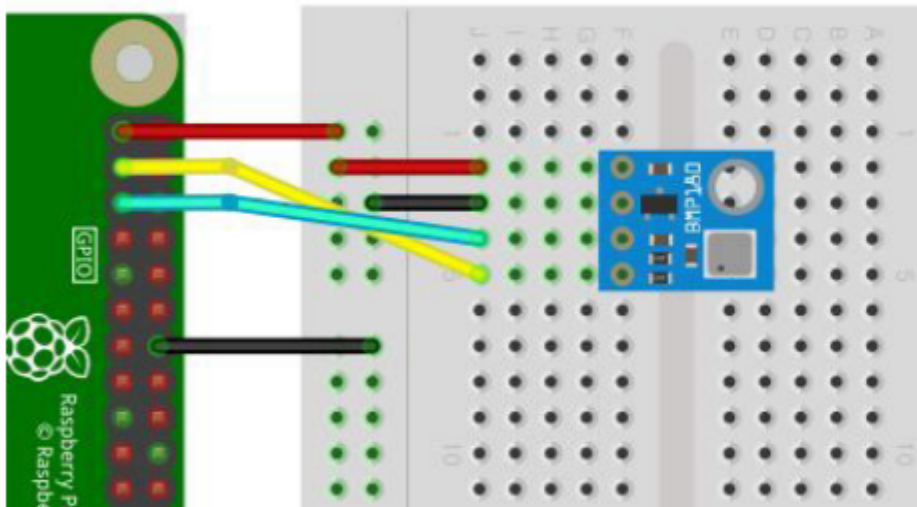
6. I²C 통신

3. 대기압 센서 (BMP 180) 실습

1) BMP180은 보쉬사에서 제작한 센서

: 기압, 온도, 고도 측정이 가능

2) 브레드보드 연결하기 (BMP180 모듈)



RP 4	BMP180
VCC(3.3V)	VIN
GND	GND
SCL (GPIO 3)	SCL
SDA (GPIO 2)	SDA

6. I²C 통신

3. 대기압 센서 (BMP 180) 실습

3) I²C에 연결된 하드웨어 주소 확인하기

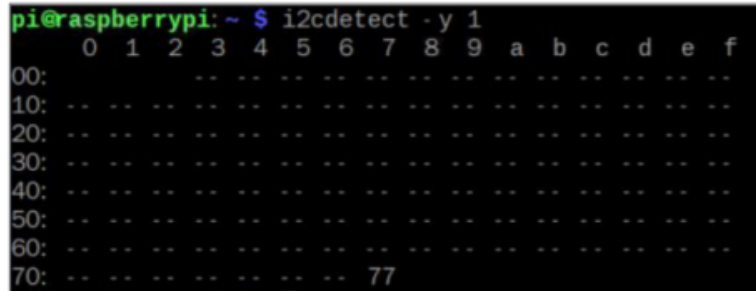
: 툴 설치

```
$ sudo apt-get install -y i2c-tools
```

: BMP 180을 연결한 후 주소(Address)를 확인

```
$ i2cdetect -y 1
```

: 주소는 0x77



```
pi@raspberrypi: ~ $ i2cdetect -y 1
 0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70:  -- -- -- -- -- -- 77 -- -- -- -- -- -- -- --
```

4) Adafruit BMP180 라이브러리 설치하기 (github에서 복사해 설치)

: \$ cd webapps

```
$ git clone https://github.com/adafruit/Adafruit_Python_BMP.git
```

```
$ cd Adafruit_Python_BMP
```

```
$ sudo python3 setup.py install
```

6. I²C 통신

3. 대기압 센서 (BMP 180) 실습

5) 파이썬 코드 작성하기

```
1  import Adafruit_BMP.BMP085 as BMP085
2
3  #BMP180 센서의 인스턴스 sensor 생성
4  sensor = BMP085.BMP085()
5
6  #온도, 압력, 고도 값을 읽어서 변수에 저장
7  temp = sensor.read_temperature()
8  pressure = sensor.read_pressure()
9  altitude = sensor.read_altitude()
10
11 #측정값 출력
12 print('Temp = {0:0.2f}*C'.format(temp))
13 print('Pressure = {0:0.2f} Pa'.format(pressure))
14 print('ALTitude = {0:0.2f} m'.format(altitude))
```

6. I²C 통신

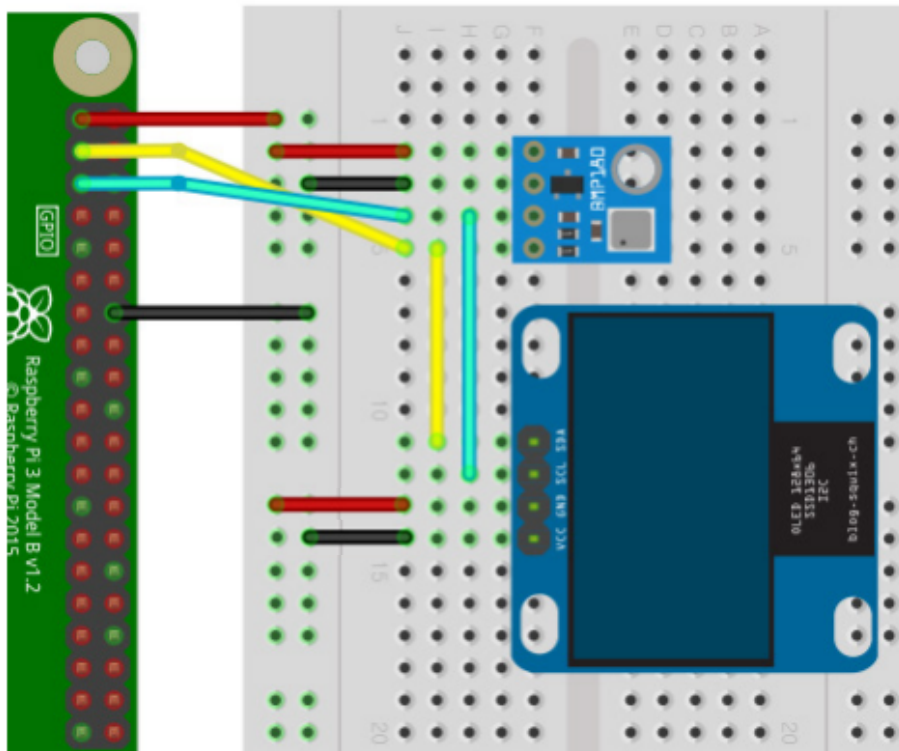
4. OLED 디스플레이 (SSD1306) 실습

1) I²C 통신방식의 디스플레이 모듈

: SCL, SDA 라인 공유

: SSD1306 드라이버 내장

2) 브레드보드 연결하기



RP 4	BMP180, OLED
VCC(3.3V)	VIN
GND	GND
SCL (GPIO 3)	SCL
SDA (GPIO 2)	SDA

6. I2C 통신

1. OLED 디스플레이 (SSD1306) -> 30분소요

1.cmd 창에서 wget "복사 링크"를 통해 다운받기

2.압축을 풀어줍니다.

3. 7zip이므로 sudo apt-get install p7zip 설치

4."7zr x 파일명"으로 압축풀어줍니다.
파일명(OLED_Module_Code.7z)

5.cmd창으로 다운받은 파일의 RaspberryPi >
python으로 이동

6.sudo python setup.py install 명령어를 통해
LIB를 설치한다.

6. I²C 통신

1. OLED 디스플레이 (SSD1306)

라즈베리파이에서

[https://www.waveshare.com/wiki/0.96inch_OLED_\(B\)](https://www.waveshare.com/wiki/0.96inch_OLED_(B))
로 접속후 이미지와 같이 Demo Codes(new)링크를 복사

Resources

- [User Manual](#)
- [Schematic](#)
- [Code](#)
- [Demo codes \(New\)](#)
- [Datasheet](#)

FAQ

Question:
What is the e
module?

- 새 탭에서 링크 열기
- 시크릿 탭에서 링크 열기
- 다른 이름으로 링크 저장...
- 링크 주소 복사
- 검사

6. I²C 통신

4. OLED 디스플레이 (SSD1306) 실습

3) I2C에 연결된 하드웨어 주소 확인하기

: BMP 180, OLED를 연결한 후 주소(Address)를 확인

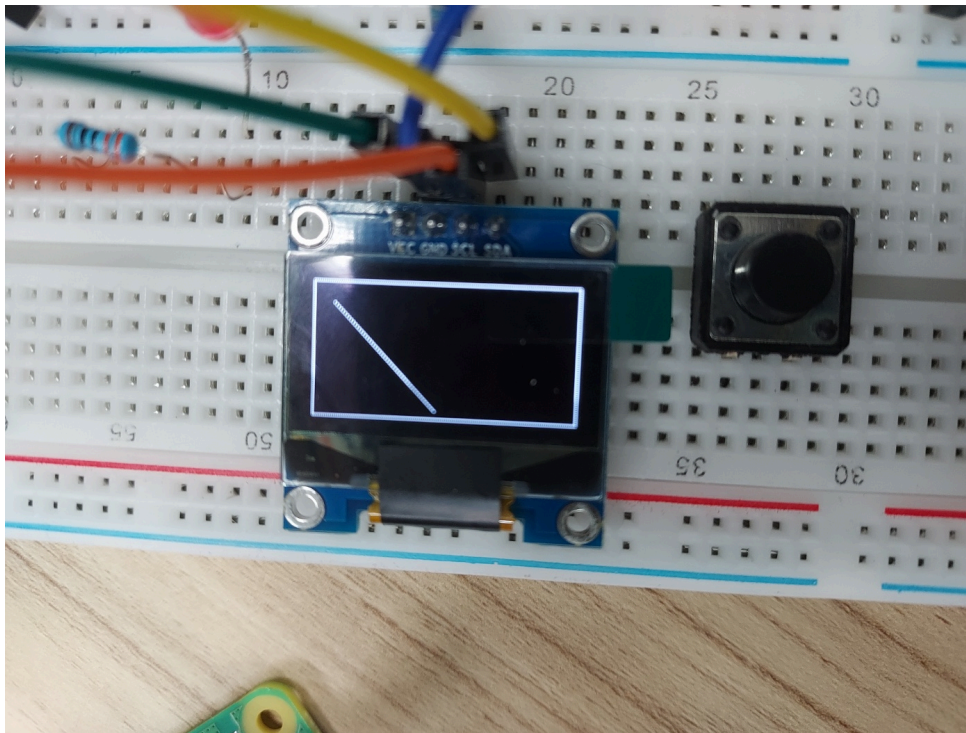
\$ i2cdetect -y 1

: BMP180 주소는 0x77, OLED 주소는 0x3c

```
pi@raspberrypi: ~ $ i2cdetect -y 1
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- 3c -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- 77 -- -- -- -- -- -- -- --
```

6. I²C 통신

1. OLED 디스플레이 (SSD1306) 박스, 라인 그리기



6. I2C 통신

1. OLED 디스플레이 (SSD1306)

```
import sys
import os
picdir = os.path.join(os.path.dirname(os.path.realpath(__file__)),
'pic')
libdir = os.path.join(os.path.dirname(os.path.realpath(__file__)),
'lib')
if os.path.exists(libdir):
    sys.path.append(libdir)

from time import sleep
from waveshare_OLED import OLED_0in96
from PIL import Image, ImageDraw, ImageFont

disp = OLED_0in96.OLED_0in96()
disp.Init()
disp.clear()

# Create blank image for drawing.
image1 = Image.new('1', (disp.width, disp.height), 255)
draw = ImageDraw.Draw(image1)
font1 = ImageFont.truetype(os.path.join(picdir, 'Font.ttc'), 18)
font2 = ImageFont.truetype(os.path.join(picdir, 'Font.ttc'), 24)

# (20,20)에 점 찍기
draw.point((20, 20))
```


6. I2C 통신

1. OLED 디스플레이 (SSD1306)

```
#draw Text
draw.text((20,0), 'Waveshare ', font = font1, fill = 0)
draw.text((20,24), u'한글 ', font = font2, fill = 0)
disp.ShowImage(disp.getbuffer(image1))
sleep(1)

#clear screen
disp.clear()
```

6. I2C 통신

1. OLED 디스플레이 (SSD1306)

```
# (20,20)에 점 찍기
draw.point((20, 20))
disp.ShowImage(disp.getbuffer(image1))
sleep(1)

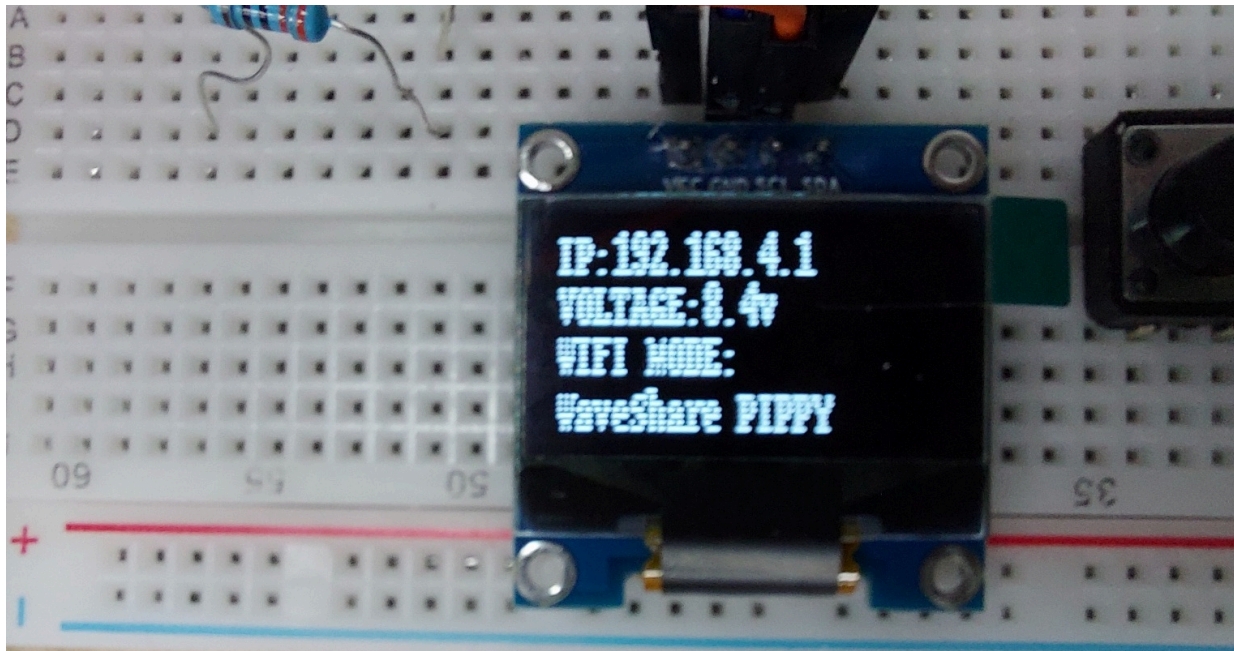
# 가장자리에 점 찍기
draw.point([(0,0),(127,0),(0,63),(127,63)])
disp.ShowImage(disp.getbuffer(image1))
sleep(1)

# 선 그리기1
for i in range(10, 60): draw.point((i, i))
disp.ShowImage(disp.getbuffer(image1))
sleep(1)

# 선 그리기 2
draw.line([(0,0),(127,0)])
draw.line([(0,0),(0,63)])
draw.line([(0,63),(127,63)])
draw.line([(127,0),(127,63)])
disp.ShowImage(disp.getbuffer(image1))
sleep(1)
```

6. I²C 통신

1. OLED 디스플레이 (SSD1306) Text 출력



6. I²C 통신

1. OLED 디스플레이 (SSD1306)

```
import time

from waveshare_OLED import OLED_0in91
from PIL import Image, ImageDraw, ImageFont

import threading

# Raspberry Pi pin configuration:
disp = OLED_0in91.OLED_0in91()
disp.Init()
disp.clear()

# Create blank image for drawing.
# Make sure to create image with mode '1' for 1-bit color.
width = disp.width
height = disp.height
image = Image.new("1", (width, height), "BLACK")
```

6. I2C 통신

1. OLED 디스플레이 (SSD1306)

```
# Get drawing object to draw on image.
draw = ImageDraw.Draw(image)

# Draw some shapes.
# First define some constants to allow easy resizing of shapes.
padding = -2
top = padding
bottom = height - padding
# Move left to right keeping track of the current x position for drawing shapes.
x = 0

# Load default font.
font = ImageFont.load_default()

text_1 = "IP:"
text_2 = "VOLTAGE:"
text_3 = "WIFI MODE:"
text_4 = "WaveShare PIPPY"
```

6. I2C 통신

1. OLED 디스플레이 (SSD1306)

```
class OLED_ctrl(threading.Thread):
    def __init__(self, *args, **kwargs):
        super(OLED_ctrl, self).__init__(*args, **kwargs)
        self.__flag = threading.Event()
        self.__flag.set()
        self.__running = threading.Event()
        self.__running.set()

    def run(self):
        while self.__running.isSet():
            self.__flag.wait()
            try:
                draw.rectangle((-1, -1, width, height), outline=0, fill=255)
                draw.text((x, top), text_1, font=font, fill=0)
                draw.text((x, top + 8), text_2, font=font, fill=0)
                draw.text((x, top + 16), text_3, font=font, fill=0)
                draw.text((x, top + 25), text_4, font=font, fill=0)

                # Display image.
                disp.ShowImage(disp.getbuffer(image))
                time.sleep(0.1)
            except:
                pass

        print("loop")
```

6. I2C 통신

1. OLED 디스플레이 (SSD1306)

```
def pause(self):
    self.__flag.clear()

def resume(self):
    self.__flag.set()

def stop(self):
    self.__flag.set()
    self.__running.clear()

def screen_show(self, position, text):
    global text_1, text_2, text_3, text_4, text_5, text_6
    if position == 1:
        text_1 = text
    elif position == 2:
        text_2 = text
    elif position == 3:
        text_3 = text
    elif position == 4:
        text_4 = text
    self.resume()
```

```
if __name__ == "__main__":
    screen = OLED_ctrl()
```